Reviewer-Directed Rebuttal Experiments

for *It Cancels: $O(r^2)$ Cholesky Updates for Whitened Operators*
NeurIPS 2026 Submission #29421

Anonymous Authors

July 25, 2026

Abstract

We report a reviewer-directed evaluation of the proposed exact $O(r^2)$ update for the double-sided whitened operator $A_{\mathrm{w}} = L^{-1}ML^{-\top}$ under rank-one changes to the underlying positive-definite metric. The evaluation comprises 145 completed configurations: 48 natural-gradient runs, 86 CPU stability and representation experiments, and 11 CUDA benchmarks. The experiments answer questions about chart-refresh sensitivity, the interpretation of predictive accuracy, regimes in which one-sided propagation is preferable, stability under extreme conditioning and near-boundary downdates, CUDA scaling, and the practical cost of rank-k extensions. A post-audit CUDA optimization follow-up, reported separately in E5, replaces the original large-rank diagnostic with a baseline-compatible parallel fallback and evaluates a corrected experimental inverse-factor representation, including held-out timing, repeated-update stability, refresh-cost, and large-rank comparisons.

The results sharpen rather than uniformly strengthen the original claims. The refresh threshold controls a clear refresh-frequency–accuracy trade-off; fixed-rank Ours and fresh refactorization have statistically overlapping error; a thin factor that remains factorized favors one-sided propagation, whereas explicit dense materialization favors the proposed two-sided representation; positive updates remain backward stable but forward error reflects the conditioning of whitening; downdate amplification exactly follows the $(1-\ell)^{-1}$ bound; the fused CUDA kernel benefits strongly from independent-update batching; and an exact prefix-scan fallback is 1.85–2.08× faster than CUDA refactorization at $r = 256$–1024. A separately labeled inverse-factor representation is 3.32–12.27× faster in held-out cached-state kernel timing at $r = 256$–1024 and 1.79–71.26× faster in a self-contained four-state ladder through $r = 65{,}536$. Its repeated-update and refresh tests show low drift for well-conditioned problems but strong condition dependence. Repeated rank-one updates are advantageous for small k, but their measured crossover occurs much earlier than a FLOP-only model predicts. We state below precisely which claims these data support and which manuscript statements should be weakened.

Scope caveat. The E1 natural-gradient sweep, including the matched-rank comparisons used to answer E2, uses a reconstructed, corrected implementation of the method described in Section 3.5. The copied artifact lacks the original Protein `.mat` file and exact submitted training driver, and its stale prototype does not reproduce submitted Setup B. Consequently, these runs are new controlled evidence about threshold sensitivity and method equivalence; they are not an exact reproduction of the submitted MSE values.

1 Scope and reviewer-question map

The Area Chair and all three reviewers considered the algebraic contribution technically sound. The experiments therefore target the reviewers' stated scope, interpretation, and empirical-support questions rather than attempting to replace the submitted evaluation. Table 1 records the purpose of each experiment.

The experiments do not directly answer fdru's notation and release questions or URn7's inverse-free and non-whitened questions. Those should be handled by textual revisions: define G, M, z, w before

Table 1: Mapping from experiments to reviewer questions and manuscript consequences.

ID	Primary question	Reviewer / meta-review	Intended manuscript use
E1	Sensitivity to the chart-refresh threshold τ_{ref}	fdru Q1; Area Chair	Add a threshold-sensitivity table and explain that τ_{ref} is an application hyperparameter, not an assumption of Theorem 1.
E2	Whether the update itself improves test MSE	RVfB Q2; Area Chair	Use the matched-rank E1 rows to establish fixed-rank equivalence and withdraw the unsupported compute-budget interpretation.
E3	When double-sided maintenance is preferable to a one-sided low-rank factorization	RVfB Q1; relevant to URn7 Q3/Q5	Add an explicit regime table: factorized thin consumers favor one-sided propagation; dense or explicitly materialized consumers favor two-sided maintenance.
E4	Numerical stability near degeneracy and the downdate boundary	URn7 Q1	Add forward- and backward-error stress tests, state the role of $\|A_{\text{w}}\|_2$, and recommend reset/guard policies.
E5	GPU scaling and the sequential nature of Givens rotations	URn7 Q2	Report supported fused-kernel correctness and batching together with the tuned, baseline-compatible parallel fallback; report corrected held-out inverse-factor timing, repeated-update and refresh evidence, and the measured large-rank ladder as a separately labeled experimental representation.
E6	Constants and practical cost for rank-k updates	URn7 Q4	Move the full $O(r^2k + rk^2 + k^3)$ cost into the main text and report measured rather than FLOP-only crossover points.

Algorithm 1; state $G, M, A_{\text{w}} \in \mathbb{R}^{r\times r}$, with G symmetric positive definite and M arbitrary; explain one-time whitening for non-whitened models; expand the inverse-free regime comparison; and give a concrete code-release plan.

2 Experimental protocol and provenance

Completion. All planned tasks completed on their first worker attempt: 48/48 natural-gradient configurations, 86/86 CPU synthetic configurations, and 11/11 CUDA configurations. The CPU and GPU worker error logs are empty. Preflight smoke tests, which use reduced data or reduced rank and overlap planned configurations, are not included in inferential tables.

Software and compute environment. All planned runs used PyTorch 2.12.0 with CUDA 12.6 on a Linux compute node. CUDA experiments ran on an NVIDIA H100 80 GB HBM3 GPU. The CPU worker allocated 100 CPU cores and 128 GB memory and launched up to ten concurrent subprocesses without per-run core isolation. Its queue interleaved the E1 natural-gradient tasks with the single-thread synthetic tasks, so an E1 run shared the allocation with a varying mixture of other worker processes. Each E1 process set PyTorch intra-operation threads to 10 and inter-operation threads to 1, with `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, and `OPENBLAS_NUM_THREADS` all set to 10. All 48 planned E1 JSON files record 10 threads, host `n22`, and Slurm job `391303`. The GPU worker allocated eight H100 GPUs and assigned one configuration to each GPU.

Core correctness checks. Every planned benchmark constructs the exact updated metric and operator and compares the incremental result with fresh Cholesky factorization and triangular solves. CPU

rank-k tests use FP64 and require relative errors below 10^{-9}. Planned CUDA tests use FP32 and require relative errors below 2×10^{-4}; all planned configurations passed. The post-audit inverse-factor follow-ups use the validation scopes stated in Section 6.2: numerical comparison with fresh results through $r = 32{,}768$, and full finite-output checks at larger measured ranks.

Summary statistics. Natural-gradient and controlled stability sweeps use three independent seeds. Reported "±" values are sample standard deviations. Microbenchmarks report median latency after warm-up; their mutable input state is prepared outside the timed interval. The representation and rank-k microbenchmarks pin each operation to one CPU thread.

3 E1: sensitivity to the refresh threshold

Reviewer response: fdru Q1 asks how gracefully the method responds across chart-refresh thresholds. The Area Chair also identifies this question as a priority.

3.1 Motivation and method

The threshold τ_{ref} triggers chart refresh when the fraction of gradient energy outside the current rank-r subspace exceeds τ_{ref}. It belongs to the transported-subspace natural-gradient application, not to the double-sided update identity itself. We used the UCI CASP/Protein regression data, an 80/20 split, ten epochs, batch size 256, and three seeds. The preconditioned first layer has 5,120 parameters. We evaluated

$$r \in \{256, 1024\}, \qquad \tau_{\text{ref}} \in \{0.01, 0.02, 0.05, 0.1, 0.2, 0.5\}.$$

The corrected optimizer starts from $G_0 = \lambda I$, $A_{\text{w}0} = 0$, uses $\lambda = 1$, and periodically refactorizes every 50 steps. At $\tau_{\text{ref}} = 0.1$, we additionally ran Ours at $r = 512$ and Refact at all three ranks. Each row in Table 2 averages three seeds.

Table 2: Refresh-threshold sensitivity. Refresh and refresh-time columns are fractions of training steps and application wall time, respectively. "Application wall" is end-to-end training time on the reconstructed, shared-worker backend; it compares a serial C++ incremental call with 10-thread PyTorch BLAS/LAPACK inside otherwise common application code and is not an isolated kernel comparison. See Table 14 for isolated single-thread timing. Drift is the maximum relative A_{w} discrepancy observed immediately before a periodic fresh reset.

Method	r	τ_{ref}	Refresh	Refresh time	Test MSE	Application wall (s)	Max. drift
Ours	256	0.01	0.668 ± 0.015	0.461 ± 0.012	0.5801 ± 0.0084	9.0 ± 0.9	1.08×10^{-6}
Ours	256	0.02	0.485 ± 0.003	0.387 ± 0.018	0.5786 ± 0.0120	8.5 ± 1.0	1.07×10^{-6}
Ours	256	0.05	0.272 ± 0.008	0.269 ± 0.007	0.5800 ± 0.0141	7.1 ± 0.3	1.39×10^{-6}
Ours	256	0.10	0.160 ± 0.003	0.168 ± 0.011	0.5842 ± 0.0162	5.6 ± 0.5	2.00×10^{-6}
Ours	256	0.20	0.091 ± 0.002	0.114 ± 0.003	0.5895 ± 0.0057	5.7 ± 0.1	2.54×10^{-6}
Ours	256	0.50	0.026 ± 0.001	0.034 ± 0.001	0.5901 ± 0.0220	5.1 ± 0.2	3.81×10^{-6}
Ours	512	0.10	0.155 ± 0.005	0.267 ± 0.011	0.5837 ± 0.0076	10.8 ± 0.7	2.51×10^{-6}
Ours	1024	0.01	0.445 ± 0.007	0.670 ± 0.006	0.5769 ± 0.0100	44.4 ± 1.7	1.87×10^{-6}
Ours	1024	0.02	0.344 ± 0.003	0.591 ± 0.005	0.5722 ± 0.0105	40.0 ± 1.4	3.15×10^{-6}
Ours	1024	0.05	0.223 ± 0.009	0.460 ± 0.005	0.5768 ± 0.0126	36.3 ± 0.1	3.60×10^{-6}
Ours	1024	0.10	0.141 ± 0.005	0.350 ± 0.013	0.5819 ± 0.0084	27.6 ± 4.0	3.17×10^{-6}
Ours	1024	0.20	0.073 ± 0.001	0.224 ± 0.012	0.5881 ± 0.0059	22.5 ± 0.4	7.95×10^{-6}
Ours	1024	0.50	0.018 ± 0.001	0.068 ± 0.002	0.5964 ± 0.0207	20.0 ± 0.7	1.96×10^{-5}
Refact	256	0.10	0.165 ± 0.007	0.166 ± 0.007	0.5820 ± 0.0091	7.0 ± 0.3	—
Refact	512	0.10	0.158 ± 0.003	0.293 ± 0.002	0.5791 ± 0.0137	9.5 ± 0.1	—
Refact	1024	0.10	0.141 ± 0.002	0.383 ± 0.005	0.5870 ± 0.0088	23.2 ± 1.5	—

3.2 Results and threshold implications

Thresholding behaves smoothly. At $r = 256$, raising τ_{ref} from 0.01 to 0.5 reduces the refresh frequency from 66.8% to 2.6% and refresh-time share from 46.1% to 3.4%. At $r = 1024$, the same change reduces refresh frequency from 44.5% to 1.8% and refresh-time share from 67.0% to 6.8%. The observed application wall-time cost of aggressive refreshing grows strongly with rank under this execution configuration.

Test-MSE intervals overlap broadly across $\tau_{\text{ref}} \in [0.01, 0.1]$. The lowest means occur at $\tau_{\text{ref}} = 0.02$, but the experiment does not establish a statistically significant optimum. Very infrequent refresh ($\tau_{\text{ref}} \in \{0.2, 0.5\}$) tends to increase MSE and reset drift. Based on the threading-independent refresh counts and the overlapping MSE intervals, $\tau_{\text{ref}} \approx 0.05$–$0.1$ is a reasonable starting range in this reconstruction: it reduces refresh events substantially without evidence of an accuracy loss. The application seconds and refresh-time shares are system- and concurrency-dependent, so a deployment should calibrate the wall-time trade-off on its own backend rather than treating these seconds as portable.

3.3 Application timing versus isolated kernel timing

The "Application wall" column in Table 2 includes the full training loop, chart projection and refresh, model forward/backward passes, preconditioning, and parameter updates. In the non-refresh core, Ours calls the serial C++ single-update path, whereas Refact calls PyTorch Cholesky and triangular-solve routines with ten intra-operation threads. Together with the shared worker conditions described in Section 2, this makes the column an application measurement, not a kernel benchmark.

The isolated comparison is the $r = 1024, k = 1$ row of Table 14. With both methods pinned to one CPU thread in FP64, the median incremental latency is 7.776 ms and the fresh-factorization latency is 70.548 ms, a 9.072× speedup. The fresh path constructs G^+ and M^+, factors G^+, and applies the two matrix triangular solves needed to whiten M^+. State cloning for the incremental path is prepared outside the timed interval.

3.4 Matched-rank equivalence

Reviewer response: RVfB Q2 asks whether the submitted "strictly better test MSE" statement can be caused by the update identity itself.

At fixed rank and with identical chart logic, an exact incremental update and fresh refactorization define the same model up to numerical and execution-order effects. The $\tau_{\text{ref}} = 0.1$ rows in Table 2 provide the matched-rank check. The refresh fractions for Ours/Refact are 0.160/0.165, 0.155/0.158, and 0.141/0.141 at $r = 256, 512, 1024$, respectively. Their final test MSEs are 0.5842 ± 0.0162/0.5820 ± 0.0091, 0.5837 ± 0.0076/0.5791 ± 0.0137, and 0.5819 ± 0.0084/0.5870 ± 0.0088. The seed variation overlaps broadly at every rank, consistent with algebraic equivalence rather than a method-induced accuracy advantage.

Because this reconstructed backend does not reproduce the submitted rank–MSE trend, these values should not replace the submitted application table. They support only the causal correction that the exact update does not itself improve predictive accuracy.

Recommended paper change: Add a compact version of Table 2, distinguish τ_{ref} from the periodic numerical reset interval, and describe rank-aware, system-specific calibration. Do not claim a universal optimal threshold. Remove "strictly better test MSE" from the abstract, introduction, and conclusion; state instead that the proposed update is algebraically equivalent to fresh refactorization at fixed rank. Do not add a new budget–accuracy claim without a separately validated experiment.

4 E3: one-sided versus two-sided representations

Reviewer response: RVfB Q1 asks when double-sided maintenance is preferable if $M = BC^\top$ is thin and a one-sided factor can be propagated. The result also sharpens the regime discussion requested by

URn7 Q3 and Q5.

4.1 Motivation and method

We form $M = BC^\top$ with $B, C \in \mathbb{R}^{r\times p}$ and compare five single-thread CPU operations after a rank-one metric update:

1. **Two-sided:** directly update dense A_{w} with the proposed $O(r^2)$ kernel.
2. **Drag:** one-sided Seeger-style propagation of $L^{-1}B$ and $L^{-1}C$, without materializing A_{w}.
3. **Drag+mat.:** drag both factors and then form their dense product.
4. **Fresh:** freshly factorize G^+ and triangular-solve for both thin factors, without materialization.
5. **Fresh+mat.:** freshly solve and then form the dense product.

We test $r \in \{256, 1024\}$, factor widths from $p = 1$ through $2r$, 30 median-timing repetitions (15 for $p > r$), and verify all representations against the same dense result. Preparation and state cloning are outside the timed region.

Table 3: Representation benchmark. Times are milliseconds. "Factor MiB" is logical FP32 storage (four bytes per value), counting only the lower triangle of the Cholesky factor plus both one-sided factors; an implementation that stores the factor densely may allocate more. Under the same convention, the dense two-sided state occupies 0.375 MiB at $r = 256$ and 6.002 MiB at $r = 1024$.

r	p	Two-sided	Drag	Drag+mat.	Fresh	Fresh+mat.	Factor MiB
256	1	0.340	0.077	0.085	0.399	0.404	0.127
256	4	0.341	0.078	0.087	0.403	0.414	0.133
256	8	0.335	0.078	0.091	0.406	0.416	0.141
256	16	0.339	0.078	0.096	0.416	0.434	0.157
256	32	0.346	0.080	0.109	0.446	0.467	0.188
256	64	0.340	0.081	0.133	0.481	0.530	0.250
256	128	0.346	0.085	0.182	0.585	0.676	0.375
256	256	0.344	0.092	0.278	0.762	0.957	0.625
256	512	0.365	0.113	0.484	1.166	1.560	1.125
1024	1	6.465	2.786	2.989	15.487	17.103	2.010
1024	4	7.911	3.049	3.523	20.651	21.113	2.033
1024	8	8.753	3.390	3.671	21.313	22.040	2.064
1024	16	9.426	3.446	4.102	22.903	23.859	2.127
1024	32	9.135	3.451	4.397	22.939	23.588	2.252
1024	64	7.429	3.017	3.985	16.899	19.123	2.502
1024	128	9.091	3.683	4.858	20.902	23.664	3.002
1024	256	8.158	3.619	6.706	25.124	29.365	4.002
1024	512	8.631	3.745	10.759	29.329	34.466	6.002
1024	1024	7.758	3.697	16.398	47.715	62.911	10.002
1024	2048	8.490	5.048	32.308	85.957	116.310	18.002

Table 4: Observed representation crossover widths. A crossover is the first tested p for which two-sided maintenance is no slower or no larger than the named alternative. "None" means no crossover in the tested range.

r	Drag	Drag+mat.	Fresh	Fresh+mat.	Memory
256	None	512	1	1	128
1024	None	512	1	1	512

4.2 Results and implications

The reviewer's thin-factor regime is real: when downstream computation can retain factorized $L^{-1}B$ and $L^{-1}C$, drag-along remains faster at every tested width, including $p = 2r$. Conversely, fresh thin solves are slower than the direct two-sided update even at $p = 1$. If a dense operator must be materialized, drag-along crosses behind the two-sided method at $p = 512$ for both ranks. The one-sided state reaches the dense-state memory footprint at exactly $p = r/2$: 128 for $r = 256$, and 512 for $r = 1024$. All relative representation discrepancies are below 6.3×10^{-7}.

The experiment covers general $M = BC^\top$ at fixed factor widths. It does not separately benchmark the symmetric specialization $M = BB^\top$, nor a single run in which p grows online. The width sweep is sufficient for the regime and memory crossover, but those unexecuted variants should not be claimed as measured.

Recommended paper change: Add a regime table stating that one-sided propagation is preferable when a fixed thin factor remains factorized throughout downstream computation. State that the proposed method is preferable when M is dense, a dense A_{w} is explicitly consumed, multiple changing factors must be combined, or factor width grows enough to erase the memory advantage. This concession directly resolves RVfB Q1 and makes the paper's scope more credible.

5 E4: conditioning, near-duplicate kernels, and downdates

Reviewer response: URn7 Q1 asks whether numerical stability degrades for nearly degenerate matrices and near the downdate boundary.

5.1 Controlled-spectrum positive updates

We generate $r = 128$ SPD matrices with log-spaced spectra and initial target condition numbers $10^2, 10^4, 10^6, 10^8$. We set $M = G - \lambda_{\min}(G)I$, so $\|A_{\mathrm{w}}\|_2 \leq 1$, apply 200 positive rank-one updates in FP32 or FP64, and compare no reset with fresh resets every 50 or 200 updates. Exact state is accumulated in FP64. Table 5 reports means over three seeds. Because positive updates increase the small eigenvalues, the condition number at step 200 can be substantially below the initial target.

Table 5: Controlled-spectrum stability. "Initial forward" is the relative A_{w} error immediately after the initial factorization. Values are means over seeds; each "max" entry is the mean of the three per-seed maxima over recorded checkpoints. $N = \infty$ denotes no reset.

Initial κ	Prec.	N	Final κ	Initial forward	Final forward	Max forward	Max operator backward
10^2	FP32	∞	9.97e+01	1.11e-06	1.07e-05	1.07e-05	2.00e-05
10^2	FP32	50	9.97e+01	1.11e-06	1.11e-06	1.32e-06	1.29e-06
10^2	FP32	200	9.97e+01	1.11e-06	1.11e-06	5.89e-06	1.05e-05
10^2	FP64	∞	9.97e+01	0	4.93e-15	4.93e-15	2.98e-15
10^2	FP64	50	9.97e+01	0	0	3.18e-15	8.13e-16
10^2	FP64	200	9.97e+01	0	0	4.20e-15	2.26e-15
10^4	FP32	∞	8.67e+03	4.21e-05	3.99e-05	4.21e-05	2.21e-05
10^4	FP32	50	8.67e+03	4.21e-05	3.80e-05	4.32e-05	1.19e-06
10^4	FP32	200	8.67e+03	4.21e-05	3.80e-05	4.21e-05	1.11e-05
10^4	FP64	∞	8.67e+03	0	1.43e-13	1.43e-13	2.64e-15
10^4	FP64	50	8.67e+03	0	0	1.28e-13	8.79e-16
10^4	FP64	200	8.67e+03	0	0	1.35e-13	1.92e-15
10^6	FP32	∞	1.25e+05	2.48e-03	5.44e-04	2.48e-03	1.79e-05
10^6	FP32	50	1.25e+05	2.48e-03	5.39e-04	2.48e-03	1.30e-06
10^6	FP32	200	1.25e+05	2.48e-03	5.39e-04	2.48e-03	1.02e-05
10^6	FP64	∞	1.25e+05	0	1.59e-12	6.62e-12	2.99e-15
10^6	FP64	50	1.25e+05	0	0	6.62e-12	9.51e-16
10^6	FP64	200	1.25e+05	0	0	6.62e-12	2.20e-15
10^8	FP32	∞	3.48e+05	1.56e-01	1.40e-03	1.56e-01	1.99e-05
10^8	FP32	50	3.48e+05	1.56e-01	1.40e-03	1.56e-01	1.20e-06
10^8	FP32	200	3.48e+05	1.56e-01	1.40e-03	1.56e-01	1.02e-05
10^8	FP64	∞	3.48e+05	0	4.25e-12	4.17e-10	2.94e-15
10^8	FP64	50	3.48e+05	0	0	4.17e-10	8.44e-16
10^8	FP64	200	3.48e+05	0	0	4.17e-10	2.02e-15

The positive-update pipeline remains backward stable: even at the most extreme FP32 setting, operator backward error stays near 2×10^{-5} without reset and near 10^{-6} with 50-step resets. Forward error, however, is condition dependent. At initial $\kappa = 10^8$, the FP32 whitened operator already has approximately 0.156 relative error before any incremental update. This is not instability caused by the orthogonal transport; it reflects the conditioning of Cholesky factorization and the two triangular solves that define whitening. FP64 remains near machine precision.

5.2 Near-duplicate RBF kernels

We next construct rank-128 RBF kernel matrices in three dimensions, replace one eighth of the inducing points by near duplicates at separations $\delta \in \{10^{-1}, \ldots, 10^{-6}\}$, and add 10^{-6} jitter. We test 100 positive updates with resets every 50 steps and three seeds. The *cumulative* operator uses $M = G - 10^{-6} I$ and satisfies $\|A_{\mathrm{w}}\|_2 \approx 1$; the *random* operator is normalized before whitening but produces an unbounded $\|A_{\mathrm{w}}\|_2$, intentionally testing the regime outside the paper's clean bounded-operator lemma.

Table 6: Near-duplicate RBF results, averaged over three seeds. Final values are recorded after the step-100 reset; "max forward" is the mean of the three per-seed trajectory maxima.

Prec.	Operator	δ	Final κ	$\|A_{\mathrm{w}}\|_2$	Final forward	Max forward	Final operator backward
FP32	cumulative	1.00e-01	4.51e+07	1.00e+00	8.48e-02	9.12e-02	1.75e-07
FP32	cumulative	1.00e-02	4.73e+07	1.00e+00	1.03e-01	1.08e-01	1.80e-07
FP32	cumulative	1.00e-03	4.73e+07	1.00e+00	1.02e-01	1.15e-01	1.84e-07
FP32	cumulative	1.00e-04	4.73e+07	1.00e+00	1.02e-01	1.09e-01	1.82e-07
FP32	cumulative	1.00e-05	4.73e+07	1.00e+00	9.46e-02	9.80e-02	1.75e-07
FP32	cumulative	1.00e-06	4.73e+07	1.00e+00	8.16e-02	8.81e-02	1.64e-07
FP32	random	1.00e-01	4.51e+07	2.22e+05	6.58e-02	8.01e-02	3.20e-05
FP32	random	1.00e-02	4.73e+07	3.11e+05	9.81e-02	1.13e-01	8.02e-04
FP32	random	1.00e-03	4.73e+07	3.59e+05	1.10e-01	1.22e-01	7.23e-03
FP32	random	1.00e-04	4.73e+07	3.61e+05	1.25e-01	1.28e-01	7.60e-04
FP32	random	1.00e-05	4.73e+07	3.61e+05	1.19e-01	1.34e-01	1.63e-05
FP32	random	1.00e-06	4.73e+07	3.61e+05	1.19e-01	1.29e-01	9.32e-06
FP64	cumulative	1.00e-01	4.51e+07	1.00e+00	0	2.61e-10	5.60e-16
FP64	cumulative	1.00e-02	4.73e+07	1.00e+00	0	3.44e-10	5.54e-16
FP64	cumulative	1.00e-03	4.73e+07	1.00e+00	0	3.44e-10	5.63e-16
FP64	cumulative	1.00e-04	4.73e+07	1.00e+00	0	3.50e-10	5.64e-16
FP64	cumulative	1.00e-05	4.73e+07	1.00e+00	0	3.54e-10	5.59e-16
FP64	cumulative	1.00e-06	4.73e+07	1.00e+00	0	3.37e-10	5.42e-16
FP64	random	1.00e-01	4.51e+07	2.22e+05	0	2.68e-10	8.05e-14
FP64	random	1.00e-02	4.73e+07	3.11e+05	0	3.49e-10	2.02e-12
FP64	random	1.00e-03	4.73e+07	3.59e+05	0	4.21e-10	1.83e-11
FP64	random	1.00e-04	4.73e+07	3.61e+05	0	4.41e-10	1.89e-12
FP64	random	1.00e-05	4.73e+07	3.61e+05	0	4.36e-10	4.36e-14
FP64	random	1.00e-06	4.73e+07	3.61e+05	0	4.12e-10	2.48e-14

The RBF experiment reinforces the controlled-spectrum result. FP32 has 8–13% forward error at $\kappa \approx 4.5$–4.7×10^7, even immediately after a reset, while the bounded cumulative operator retains backward error near 2×10^{-7}. For arbitrary random M, $\|A_w\|_2$ grows to 2.2–3.6 $\times 10^5$; the bounded-operator premise no longer holds, and FP32 operator backward error can reach 7.2×10^{-3}. FP64 remains accurate in both cases.

5.3 Near-boundary downdates

For a downdate $G^- = G - zz^\top$, define leverage $\ell = z^\top G^{-1} z < 1$. Appendix C.4 predicts worst-direction amplification $(1-\ell)^{-1}$. We evaluate $\ell \in \{0.1, 0.5, 0.9, 0.99, 0.999\}$ at $r \in \{64, 256\}$, three seeds, and compare the constructed worst direction with random FP32 roundoff. The final column gives the outer-product weight β in $G - \beta zz^\top$ that enforces a guard $\beta\ell \le 0.5$; it is not a multiplier on z itself (the equivalent vector multiplier is $\sqrt{\beta}$).

Table 7: Downdate amplification. Random-roundoff entries are mean ± sample standard deviation over three seeds.

r	ℓ	Bound	Worst observed	FP32 roundoff	Guard weight β
64	0.1	1.11	1.11	1.00 ± 0.0017	1.0000
64	0.5	2	2	1.04 ± 0.022	1.0000
64	0.9	10	10	1.93 ± 0.056	0.5556
64	0.99	100	100	8.19 ± 1.4	0.5051
64	0.999	1000	1000	55.7 ± 20	0.5005
256	0.1	1.11	1.11	1.00 ± 0.00014	1.0000
256	0.5	2	2	1.01 ± 0.012	1.0000
256	0.9	10	10	1.81 ± 0.089	0.5556
256	0.99	100	100	6.41 ± 1.2	0.5051
256	0.999	1000	1000	32.1 ± 16	0.5005

The observed worst-direction amplification matches the theoretical bound to numerical precision at every rank and leverage. Typical roundoff grows more slowly than the worst case, but rises sharply as $\ell \to 1$. Thus the downdate warning is not merely theoretical.

Recommended paper change: State that the orthogonal transport is nonexpansive, but the full pipeline inherits the conditioning of factorization and triangular solves. Report backward error alongside forward error. Recommend FP64, additional jitter, or more frequent resets for extremely ill-conditioned whitening. For downdates, expose leverage and either reject, damp to a specified threshold, or refactorize as ℓ approaches one.

6 E5: CUDA correctness, batching, and optimized fallback

Reviewer response: URn7 Q2 asks whether sequential Givens rotations limit GPU parallelism and how performance scales with rank.

6.1 Method and algebraic equivalence

On an H100 80 GB GPU, we compare the Triton update with a fair PyTorch baseline that forms $G^+ = G + zz^\top$, $M^+ = M + zw^\top$, computes a batched Cholesky factorization, and performs two batched triangular solves. CUDA events measure median device time after warm-up. The fused kernel is tested at $r \in \{64, 128\}$ and independent-update batches $T \in \{1, 4, 16, 64\}$. A follow-up sweep replaces the original serial tiled fallback at $r \in \{256, 512, 1024\}$, $T = 1$, with a baseline-compatible parallel path.

"Baseline-compatible" means that it maintains exactly the same L and A_{w} state as the original method; it does not cache an inverse factor or change the mathematical output.

The main speedup comes from exposing parallelism already present in the original algebra. Let

$$u = L^{-1}z, \qquad y = L^{-1}w, \qquad S_i = 1 + \sum_{j=0}^{i} u_j^2, \quad S_{-1} = 1.$$

Because

$$G^{+} = L(I + uu^{\top})L^{\top}, \qquad C = \mathrm{chol}(I + uu^{\top}), \qquad L^{+} = LC,$$

the original sequence of Givens rotations has the closed-form coefficients

$$c_i = \sqrt{\frac{S_{i-1}}{S_i}}, \qquad s_i = \frac{u_i}{\sqrt{S_i}}.$$

The two vectors required by the rank-one operator correction are likewise prefix expressions:

$$(C^{-1}u)_i = \frac{u_i}{\sqrt{S_{i-1}S_i}}, \qquad (C^{-1}y)_i = c_i \left(y_i - \frac{u_i}{S_{i-1}} \sum_{j<i} u_j y_j \right).$$

Thus the implementation obtains u and y in one two-right-hand-side triangular solve and obtains every $c_i, s_i, (C^{-1}u)_i, (C^{-1}y)_i$ with parallel prefix sums.

The factor update is also a row-wise scan. Writing $v_z = C^{-1}u$, for $i \geq j$,

$$L^{+}_{ij} = \frac{L_{ij}}{c_j} + (v_z)_j \sum_{k=j+1}^{i} L_{ik}u_k.$$

Each row therefore uses an independent reverse cumulative sum rather than an r-step mutation chain. Finally,

$$A^{+}_{\mathrm{w}} = C^{-1}A_{\mathrm{w}}C^{-\top} + (C^{-1}u)(C^{-1}y)^{\top}.$$

Each left or right Givens sweep has the affine recurrence $p_i = c_i p_{i-1} - s_i x_i$, with output $x'_i = c_i x_i + s_i p_{i-1}$. Associative composition of these affine maps is an exact prefix scan in real arithmetic. The CUDA path therefore reorders, but does not approximate or omit, the original update; its only difference is the usual FP32 reassociation error. Reusable workspaces, parallel row/column tiles, and fusion of the final outer product into the right sweep remove the remaining allocator and launch overheads.

Table 8: CUDA results on an H100 80 GB. Latency is milliseconds; throughput is updates/s; memory is reusable workspace MiB. "BW" is a conservative minimum effective bandwidth rather than peak hardware bandwidth.

Impl.	r	T	Ours ms	Refact ms	Ours/s	Refact/s	Speedup	BW GB/s	Workspace MiB	A_{w} rel. err.
fused	64	1	0.106	0.125	9,445	8,011	1.179	0.64	0.018	2.22e-07
fused	64	4	0.106	0.222	37,730	18,052	2.090	2.55	0.067	1.94e-07
fused	64	16	0.107	0.199	150,173	80,308	1.870	10.15	0.270	2.18e-07
fused	64	64	0.107	0.206	599,251	310,752	1.928	40.50	1.078	2.18e-07
fused	128	1	0.189	0.172	5,288	5,815	0.909	1.41	0.065	2.55e-07
fused	128	4	0.189	0.368	21,176	10,877	1.947	5.64	0.260	2.03e-07
fused	128	16	0.187	0.330	85,368	48,466	1.761	22.73	1.039	2.32e-07
fused	128	64	0.190	0.368	337,553	173,769	1.943	89.87	4.156	2.32e-07
parallel	256	1	0.154	0.285	6,487	3,508	1.849	6.86	0.509	2.72e-07
parallel	512	1	0.267	0.522	3,752	1,918	1.957	15.80	2.018	2.61e-07
parallel	1024	1	0.496	1.033	2,015	968	2.082	33.87	8.035	3.08e-07

All configurations are accurate: relative L error is 0.9–1.4 $\times 10^{-7}$ and relative A_{w} error is 1.9–3.1 $\times 10^{-7}$. The fused path demonstrates the value of independent-update batching. At $T = 1$, $r = 128$ is slightly slower than REFACT; at $T = 4$–64, fused speedups are 1.76–2.09×. The baseline-compatible parallel fallback is also faster for every tested large rank: 1.85×, 1.96×, and 2.08× at $r = 256, 512, 1024$, respectively. At $r = 1024$, it reduces the old tiled fallback's 2.798 ms diagnostic to 0.496 ms, a 5.64× implementation speedup, without changing maintained state. Phase profiling identifies the vendor two-right-hand-side triangular solve as the remaining bottleneck (approximately 0.44 ms at $r = 1024$); padding the solve to wider right-hand sides did not improve it.

6.2 Experimental inverse-factor state and refresh policy

We separately tested a more aggressive representation that stores L^{-1}, rather than L, between refreshes. It uses the exact identity

$$(L^+)^{-1} = C^{-1}L^{-1}, \qquad \left[(L^+)^{-1}\right]_{ij} = c_i \left((L^{-1})_{ij} - \frac{u_i}{S_{i-1}} \sum_{k<i} u_k (L^{-1})_{kj} \right),$$

so future u and y are matrix–vector products and the inverse-factor update is another column-wise prefix scan. This preserves the update algebra, but changes the stored factor representation and therefore requires separate timing, drift, refresh, and memory evidence.

The first inverse-factor timing and stability jobs were invalidated because `torch.linalg.inv` produced column-major-strided inputs while the Triton kernel's contract is a row-major contiguous lower triangle. Every result below comes from corrected initialization that explicitly takes the lower triangle and makes it contiguous. Invalid jobs 393306 and 393301 remain excluded.

6.2.1 Held-out cached-state kernel timing

The single-block configurations were selected using seed 0 and then frozen. Job 393470 evaluated three held-out problem seeds (1–3); each seed-level latency is a CUDA-event median over 60 prepared repeats. Table 9 reports the mean and sample standard deviation across those three seed-level medians, and the maximum correctness error across the held-out seeds.

Table 9: Corrected held-out single-block inverse-factor timing on one H100. The incremental interval updates cached L^{-1} and A_{w}; initial inverse construction, refresh, and maintenance of exact G, M are excluded. REFACT forms G^+, M^+, factorizes, and rewhitens. Thus this is an isolated cached-state kernel comparison, not a self-contained application measurement.

r	Inverse ms	Refact ms	Speedup	L^{-1} rel. err.	A_{w} rel. err.
256	0.08349 ± 0.00056	0.27738 ± 0.00132	3.322 ± 0.021	1.86e-07	2.73e-07
512	0.08308 ± 0.00061	0.50752 ± 0.00189	6.109 ± 0.041	1.82e-07	2.63e-07
1024	0.08364 ± 0.00029	1.02632 ± 0.00168	12.271 ± 0.053	2.05e-07	3.04e-07

The held-out result establishes a large isolated-kernel advantage for a consumer that already has the inverse-factor state: 3.32× at $r = 256$, 6.11× at $r = 512$, and 12.27× at $r = 1024$. It does not by itself establish the amortized cost of constructing or refreshing that state; the refresh and self-contained comparisons below measure those costs separately.

6.2.2 Repeated updates and condition dependence

Job 393428 applies 500 updates without reset to three well-conditioned synthetic problems per row. Table 10 reports the median final error over seeds. The first three rows use the single-block kernel; the last row independently checks the in-place recurrence used by the large-rank path.

Table 10: Corrected 500-update FP32 inverse-factor stability. Values are medians over three seeds; the initial measured condition numbers are near five.

Path	r	Initial $\kappa(G)$	L^{-1} rel. err.	A_{w} rel. err.	Inverse residual
single-block	256	4.91	6.80e-06	1.21e-05	6.91e-06
single-block	512	4.98	1.01e-05	1.21e-05	1.06e-05
single-block	1024	4.99	7.65e-06	1.26e-05	7.75e-06
in-place	1024	4.99	7.65e-06	1.45e-05	7.75e-06

The well-conditioned errors grow gradually and remain below 1.5×10^{-5} after 500 updates. This is not a condition-independent guarantee. Job 393454 fixes $r = 256$, applies 200 updates, and varies the initial condition number. As Table 11 shows, the operator error rises to about 4.1% at $\kappa(G) \approx 10^6$, consistent with the forward-conditioning mechanism isolated in E4.

Table 11: Condition stress for the corrected single-block inverse-factor path at $r = 256$ after 200 FP32 updates. Entries are medians over three seeds.

Target $\kappa(G)$	Measured $\kappa(G)$	L^{-1} rel. err.	A_{w} rel. err.	Inverse residual
10^2	1.00×10^2	3.06e-06	5.25e-06	3.08e-06
10^4	1.00×10^4	1.20e-04	1.78e-04	6.80e-05
10^6	1.00×10^6	2.32e-02	4.11e-02	8.98e-03

6.2.3 Refresh cost and drift

Job 393472 measures 200-update sequences at refresh intervals $q \in \{10, 50, 100, 200\}$, plus no refresh, for three seeds at each rank. Updates to exact G and M are inside every timed step. At a refresh, the timed region also includes Cholesky factorization, construction of L^{-1} by a triangular solve, and complete rewhitening of M. REFACT performs these operations every step. Initial state construction is outside both timed sequences.

Table 12 gives the median CUDA-event time per update and median speedup across seeds. The error columns are the worst errors over seeds measured immediately *before* a refresh (or at the final unrefreshed tail), rather than the potentially zero post-refresh final error. The median fresh-every-step baselines are 0.251, 0.478, and 0.983 ms/update at $r = 256, 512, 1024$, respectively.

All tested refresh policies remain faster than refactorizing every step. Refreshing every 10 steps holds the worst observed A_{w} error below 4.8×10^{-7} but pays the highest refresh cost. An interval of 50 steps keeps the worst observed A_{w} error below 1.5×10^{-6} while retaining speedups of 2.43×, 4.45×, and 8.46× at the three ranks. These are controlled synthetic sequences, so $q = 50$ is an illustrative cost–drift point, not a universal policy; conditioning and the application's error tolerance should determine the reset interval. This numerical refresh is distinct from the chart-refresh threshold τ_{ref} in E1.

6.2.4 Self-contained large-rank comparison

The full-axis in-place path removes the single-block rank limit while retaining exact G, M for future refresh. The comparison maintains four dense FP32 state matrices on both sides: $G, M, L^{-1}, A_{\mathrm{w}}$ for the inverse-factor path and G, M, L, A_{w} for REFACT. Every timed event includes the G, M rank-one outer updates. Each repeat prepares the same well-scaled state outside timing and measures one recurrent-compatible update, so this is not a long-horizon stability experiment.

Tile and warp choices were selected at $r = 8192$ and $r = 32768$ in job 393466; none of those tuning latencies enters Table 13. The final measurement ladder is job 393491 through $r = 49{,}152$, with the

Table 12: Periodic inverse-factor refresh over 200 updates. Refresh work and exact G, M maintenance are included in event timing. q = none means no refresh.

r	q	Refreshes	Event ms/update	Speedup	Max. pre-refresh L^{-1} err.	Max. pre-refresh A_w err.
256	none	0	0.10169	2.474	2.81e-06	4.92e-06
256	10	20	0.12165	2.076	2.88e-07	4.59e-07
256	50	4	0.10371	2.432	7.82e-07	1.37e-06
256	100	2	0.10001	2.507	1.46e-06	2.56e-06
256	200	1	0.09979	2.513	2.81e-06	4.92e-06
512	none	0	0.10043	4.759	4.12e-06	4.97e-06
512	10	20	0.13606	3.506	3.21e-07	4.36e-07
512	50	4	0.10746	4.453	1.11e-06	1.37e-06
512	100	2	0.10246	4.665	2.12e-06	2.57e-06
512	200	1	0.10074	4.758	4.12e-06	4.97e-06
1024	none	0	0.09928	9.879	3.12e-06	5.15e-06
1024	10	20	0.19795	4.966	3.02e-07	4.76e-07
1024	50	4	0.11615	8.463	8.66e-07	1.42e-06
1024	100	2	0.10900	8.993	1.62e-06	2.65e-06
1024	200	1	0.10504	9.344	3.12e-06	5.15e-06

$r = 65{,}536$ row completed by job 393509. The implementation is compared numerically with a fresh factorization through $r = 32{,}768$. At larger ranks, memory-safe validation checks every output element for finiteness but does not compute a fresh numerical error.

Table 13: Final self-contained large-rank ladder on one H100 80 GB. Times are CUDA-event medians. Both methods maintain four dense matrices; "State" is their minimum FP32 storage. Fresh-check entries report L^{-1}/A_w relative errors.

r	Inverse ms	Refact ms	Speedup	State GiB	Validation
2,048	1.423	2.549	1.79	0.0625	fresh: 3.90e-08 / 9.79e-08
4,096	3.560	7.699	2.16	0.25	fresh: 5.01e-08 / 1.24e-07
8,192	7.949	39.835	5.01	1.00	fresh: 3.76e-08 / 9.83e-08
16,384	19.855	246.991	12.44	4.00	fresh: 4.64e-08 / 1.24e-07
32,768	57.899	1,760.636	30.41	16.00	fresh: 3.39e-08 / 1.04e-07
49,152	112.901	5,717.335	50.64	36.00	full finite only
65,536	183.576	13,081.015	71.26	64.00	full finite only

The measured crossover is already favorable at $r = 2048$, and the speedup grows with rank as the quadratic inverse-factor update is compared with cubic factorization and dense triangular solves. At $r = 65{,}536$, the final event medians are 183.576 ms versus 13.081 s, a 71.26× speedup. This largest row is a full finite-output check, not a fresh numerical-correctness check, and the 64 GiB four-matrix state nearly fills the device. We therefore make no claim beyond the measured range.

Recommended paper change: Use the fused batching results and the baseline-compatible parallel rows of Table 8 to answer URn7 Q2, and explain the exact prefix-scan reformulation. Present Tables 9–13 separately as evidence for an experimental inverse-factor state: distinguish isolated cached-state timing from self-contained timing, report the condition-dependent drift and measured refresh trade-off, and limit large-rank claims to the validation scope stated in the table.

7 E6: practical rank-k cost

Reviewer response: URn7 Q4 asks about constants when extending rank-one updates to rank-k changes.

7.1 Motivation and method

For $Z, W \in \mathbb{R}^{r\times k}$, we compare k sequential calls to the FP64 single-thread rank-one CPU kernel with one fresh factorization of

$$G^+ = G + ZZ^\top, \qquad M^+ = M + ZW^\top.$$

We test $r \in \{256, 1024\}$, $k \in \{1, 2, 4, 8, 16, 32\}$, use median latency over 20 repetitions (10 for $k > 8$), and verify against fresh FP64 factorization. No native blocked rank-k implementation is benchmarked.

Table 14: Practical rank-k CPU results. Latencies are milliseconds. Speedup is REFACT time divided by sequential rank-one time.

r	k	Sequential	Refact	Speedup	A_w rel. error
256	1	0.418	1.150	2.751	5.12e-16
256	2	0.813	1.134	1.394	5.40e-16
256	4	1.635	1.139	0.697	5.62e-16
256	8	3.212	1.133	0.353	6.03e-16
256	16	6.343	1.196	0.189	6.97e-16
256	32	12.642	1.210	0.096	8.43e-16
1024	1	7.776	70.548	9.072	6.20e-16
1024	2	19.908	84.001	4.219	6.33e-16
1024	4	44.213	92.219	2.086	6.56e-16
1024	8	90.744	74.964	0.826	6.98e-16
1024	16	159.037	77.950	0.490	7.74e-16
1024	32	303.179	75.154	0.248	9.14e-16

7.2 Results and implications

Sequential rank-one updates are exact to FP64 precision and valuable for small k. At $r = 256$, the measured crossover lies between $k = 2$ and $k = 4$; at $r = 1024$, it lies between $k = 4$ and $k = 8$. This is much earlier than the FLOP-only predictions $k \approx 33.2$ and $k \approx 132.7$, respectively. The discrepancy reflects optimized cubic BLAS/LAPACK kernels, cache behavior, and repeated rank-one call overhead. It does not contradict the asymptotic cost, but it invalidates a universal practical crossover derived solely from flop counts. A blocked compact-WY/Householder or blocked Givens implementation may move the crossover, but was not run.

Recommended paper change: State the complete rank-k complexity $O(r^2k+rk^2+k^3)$, reducing to $O(r^2k)$ only for $k \ll r$. Report the strong $k = 1$ speedups and the measured repeated-update crossover. Present the $k \approx 0.13r$ calculation only as an arithmetic model, not a wall-clock prediction, and identify a native blocked rank-k kernel as future work.

8 Integrated manuscript and rebuttal recommendations

8.1 Reviewer RVfB

- **Q1, one-sided alternatives.** Agree explicitly that a fixed, thin factor retained in factorized form is a regime where one-sided propagation is preferable. Cite the absence of a drag-only crossover in Table 4. Explain that direct two-sided maintenance is advantageous when an explicit dense operator is required: it beats fresh solves at the first tested width and beats drag-plus-materialization by $p = 512$.

8.2 Reviewer fdru

- **Q1, refresh threshold.** Report the smooth refresh-frequency curve from E1. Use the refresh counts and MSE intervals to suggest $\tau_{\text{ref}} \approx 0.05$–$0.1$ as a starting range, while requiring system-specific timing calibration rather than treating the E1 seconds as portable. Clarify that this threshold is application-specific and distinct from a periodic numerical reset.
- **Q2, dissemination.** Give an anonymized release plan with C++, Triton, and MLX build instructions, correctness tests, supported software/hardware, and a public-release date. Treat AMD and Intel kernels as future work, consistent with the reviewer's statement that they are not required.
- **Notation.** Introduce z as the metric-update vector and w as the operator-update vector before Algorithm 1.

8.3 Reviewer URn7

- **Q1, stability.** Distinguish forward conditioning from backward stability. The orthogonal transport is stable, FP64 is robust, and resets control accumulated backward error; FP32 whitening can still have large forward error when $\kappa(G)$ is extreme. Expose and guard downdate leverage.
- **Q2, GPU scaling.** Report fused FP32 correctness and batching speedups together with the tuned, baseline-compatible prefix-scan fallback. Separately report the corrected held-out inverse-factor timing, condition-dependent 500-update drift, refresh-cost sweep, and self-contained large-rank ladder. Keep this representation explicitly experimental, distinguish fresh numerical checks from finite-only checks, and state its refresh and four-matrix memory requirements.
- **Q3, inverse-free methods.** Add a textual regime comparison: inverse-free methods are attractive for batch-oriented, matrix-multiply workloads; the proposed method is for systems that already maintain a sequential Cholesky state and consume whitened quantities.
- **Q4, rank-k.** Add the full complexity and the measured small-k crossover from E6.
- **Q5, whitening.** Explain that a non-whitened model pays a one-time coordinate transformation; maintenance is worthwhile only when downstream computation repeatedly consumes the whitened operator.

8.4 Area Chair

The integrated response should emphasize six concrete changes: (i) a one-sided/two-sided regime table; (ii) corrected fixed-rank MSE wording, supported by the matched-rank Table 2 rows rather than a budget-truncated comparison; (iii) the E1 threshold sweep; (iv) forward and backward stability evidence plus a downdate guard; (v) supported fused CUDA batching and baseline-compatible parallel-fallback results, with the corrected inverse-factor timing, refresh, and large-rank evidence separated as an experimental stored-state representation; and (vi) the complete rank-k complexity with measured constants. Independently, the abstract and introduction should state explicitly that $G, M, A_{\text{w}} \in \mathbb{R}^{r\times r}$, G is SPD, and M may be arbitrary.

9 Conclusion

The reviewer-directed experiments support the paper's central message while substantially improving its precision. They confirm exactness and numerical agreement, identify a practical threshold range, validate the one-sided regime raised by RVfB, demonstrate backward stability and exact downdate

amplification, and show that fused CUDA batching is effective. They also show that the exact prefix-scan fallback is faster than CUDA refactorization through rank 1024. The corrected inverse-factor follow-up adds a condition-aware refresh policy and a self-contained measured ladder from $r = 2048$ to $r = 65{,}536$, where the final finite-checked row is $71.26\times$ faster than refactorization. It also identifies the limits of that result: the representation changes stored state, drift grows sharply under extreme conditioning, and fresh numerical correctness is checked only through $r = 32{,}768$. More broadly, the update does not itself improve MSE, extreme FP32 whitening is not condition-number independent, and practical rank-k crossover is not justified by asymptotic flop counts alone. Incorporating these distinctions would make the revision more credible, easier to apply, and more directly responsive to all reviewer questions.

A Result provenance and regeneration

The tables in this report are derived from the planned-task outputs under `rebuttal_results/`. The canonical machine-readable artifact is the curated set of individual JSON files, which retain seeds, repeat counts and, where recorded, raw timing samples, environment metadata, full training histories, and checkpoint diagnostics. The task definitions are in `rebuttal_experiments/task_matrix.py`; a completion audit and summary can be regenerated with `rebuttal_experiments/aggregate.py`.

The original scientific tables include exactly the 145 planned worker outputs. E5 additionally reports the post-audit CUDA follow-ups under `rebuttal_results/cuda/` and `rebuttal_results/cuda_tuning/`: parallel tuning job 393048, phase profiles job 393047, production-path stability job 393321, and RHS width/orientation probes jobs 393087 and 393161; corrected CUDA test job 393425; inverse-factor configuration-selection job 393427 and held-out job 393470; 500-update stability job 393428; condition-stress job 393454; refresh job 393472; and the final large-rank measurement jobs 393491 and 393509. Large-rank tuning job 393466 selected tile and warp configurations but contributes no claimed latency. Invalidated layout-bug jobs 393306 and 393301 are explicitly excluded. Files whose names contain `smoke`, tuning rows, and early reduced-rank diagnostics remain excluded from inferential summaries and final speedup claims.

B Limitations of the new evidence

- The E1 sweep and its matched-rank comparison for E2 use a reconstructed natural-gradient backend and do not reproduce submitted Setup B; they should not replace the submitted application results.
- E3 evaluates general fixed-width $M = BC^\top$, not the symmetric specialization or a single online-growing-factor trajectory.
- E4 uses synthetic controlled spectra and RBF kernels. It isolates numerical mechanisms but is not an application-level failure-rate study.
- E5's production-compatible comparison is limited to one H100, FP32, batch one, and $r \leq 1024$. The inverse-factor path changes stored state and is evaluated on synthetic updates: held-out timing uses only three problem seeds, repeated-update and refresh validation stop at $r = 1024$, and the large-rank ladder measures one prepared update per repeat. Periodic refresh cost is included only in the $r \leq 1024$ refresh sweep. The four-state large-rank comparison checks numerical agreement with fresh results through $r = 32{,}768$, but checks only full-output finiteness at $r = 49{,}152$ and 65,536; the latter requires 64 GiB of matrix state.
- E6 compares repeated rank-one calls with refactorization; it does not evaluate a native blocked rank-k kernel.